



南京航空航天大学
NANJING UNIVERSITY OF AERONAUTICS AND ASTRONAUTICS

《大模型安全与知识增强》大作业报告

选 题： 03-KnowledgeEditing

题 目： 大模型知识编辑：单事实编辑、批量编辑与综合评估

院 系： 计算机科学与技术学院/软件学院

专 业： 计算机科学与技术

姓 名： 王子璇

学 号： SX2516003

2026 年 5 月 10 日

摘要

本实验围绕大模型知识编辑任务展开，目标是在不进行完整预训练或全量微调的条件下，对模型已有事实进行定点更新，并通过编辑成功率、泛化性和局部性三类指标判断编辑操作是否真正有效。课程仓库要求方向 03 完成四项基本任务：首先构建 10 条事实更新数据并进行编辑前测试，其次完成单条事实编辑实践，再次完成 500 条数据的批量编辑实践，最后汇总 ES、PS、NS 并给出终端截图、指标表和失败案例分析。本文按照这一顺序组织实验材料和报告结构，尽量让每一个任务都有对应的数据文件、运行脚本、终端截图、结构化结果和文字解释。

当前结果显示，Baseline 阶段共测试 10 条自定义事实，旧知识命中率为 100.0%，编辑前新知识命中率为 0.0%，说明样本在编辑前具有明确的旧知识状态。ROME 风格单事实编辑阶段对 10 条事实逐条处理，每条事实在编辑前重置模型状态，最终 ES 为 100.0%，PS 为 90.0%，NS 为 100.0%。其中 PS 未达到满分，主要来自 The Great Gatsby 和 Python 两个样本的同义改写提示未全部命中新知识。MEMIT 风格批量编辑阶段一次性处理 500 条批量样本，ES、PS、NS 均为 100.0%，说明当前流程能够稳定完成批量写入、评估和结果汇总。

除基础任务外，报告还加入 SimpleRAG 附加对照，用于比较参数化编辑和非参数化检索式知识增强在更新方式、响应延迟、可撤回性和结果稳定性方面的差异。本文的指标、截图和表格均与项目目录中的运行记录、JSON 文件和 CSV 文件保持一致，便于复核结果。

关键词：知识编辑；ROME；MEMIT；EasyEdit；Efficacy；Generalization；Locality；RAG 对照

一、作业要求与实验目标

课程仓库对 03-KnowledgeEditing 方向的定位比较明确：知识编辑用于处理大语言模型知识过期、错误事实和事实冲突等问题。与重新训练或全量微调相比，知识编辑希望用较低代价更新某些具体事实，同时尽量不改变模型在其他问题上的行为。这样的实验不只关注某个提示词是否能返回新答案，还需要同时观察改写问法能否泛化，以及无关知识是否受到误伤。

按照 GitHub 中的方向说明，本实验至少需要完成四个任务。Task 1 是编辑前测试，需要构建 10 条事实更新数据，并证明模型在编辑前仍处于旧知识或知识盲区状态。Task 2 是单条事实编辑，需要基于 ROME 思想逐条注入新事实，并且每次编辑前重置模型状态。Task 3 是批量知识编辑，需要选取 500 条数据并执行批量写入，同时记录耗时和资源占用。Task 4 是综合评估，需要计算 ES、PS 和 NS 三项指标，并用表格形式汇总百分比结果。报告还必须包含 Task 1 到 Task 4 的终端输出截图，并对 MEMIT 批量编辑效果和失败案例进行分析。

为了让报告和课程要求形成明确对应关系，本文先给出作业要求与实现文件的对应表，再依次展开数据构建、指标定义、四个任务的实验过程和结果分析。每一节都先说明该任务在知识编辑实验中的作用，再展示对应的运行结果，最后解释结果能够说明什么。这样写的好处是，读者不仅能看到最终数值，还能追溯这些数值对应的代码文件、结果文件和截图。

本报告对“结果”的理解分为两个层次。第一层是课程交付层面，重点是是否完成完整的运行链路、结构化指标和截图材料。第二层是研究判断层面，重点是 ES、PS、NS 三类指标之间是否一致，失败案例暴露了什么，以及参数化编辑与 RAG 对照各自适合什么场景。本文不会只把 100% 的指标作为结论，而是将 ROME 阶段 PS 下降的样本单独分析，以说明知识编辑中的直接成功和泛化成功并非同一个问题。

项目要求	GitHub 要点	对应文件	本文落实情况
Task 1	构建 10 条事实更新数据，并在编辑前记录旧知识与新知识命中情况	baseline.py; fact_updates_10.jsonl; baseline_results.json	已完成，旧知识命中率 100.0%，编辑前新知识命中率 0.0%
Task 2	逐条执行单事实编辑，每次重置模型状态，并计算 ES、PS、NS	edit_rome.py; rome_results.json; rome_case_metrics.csv	已完成，ES 100.0%，PS 90.0%，NS 100.0%
Task 3	对 500 条数据执行批量编辑，并记录耗时与资源占用	edit_memit.py; memit_batch_500.jsonl; memit_results.json	已完成，500 条样本 ES、PS、NS 均为 100.0%
Task 4	汇总 ES、PS、NS 百分比，给出终端截图、表格与失败案例	evaluate.py; summary.csv; failure_cases.csv; figures	已完成，报告中插入终端截图、指标表和失败案例表
Bonus	构建 RAG 对照，比较参数化编辑和检索式知识增强差异	rag_compare.py; rag_compare.csv; rag_compare.json	已完成，作为附加实验放在第 10 节

表 1 GitHub 作业要求与本文实现对应关系

表 1 可以看出，本文将课程要求拆解为数据、代码、结果文件和报告内容四个层次。这样处理后，任务要求、运行命令、结构化结果和报告截图之间形成清晰对应关系。

方向-03：大模型知识编辑（Knowledge Editing for LLMs）

Task 1	构建 10 条事实更新数据，记录编辑前回答，证明存在旧知识或知识盲区。
Task 2	基于 ROME 思路逐条编辑 10 条事实，每条事实编辑前重置状态。
Task 3	使用 MEMIT 思路对 500 条数据进行批量编辑，并记录耗时和资源占用。
Task 4	计算 ES、PS、NS，分别评估编辑成功率、泛化性和局部性。
Deliverables	提交 baseline.py、edit_rome.py、edit_memit.py、evaluate.py、README 和报告。
Report	报告包含 Task 1-4 终端截图、ES/PS/NS 表、MEMIT 分析和失败案例。

图 1 GitHub 作业要求摘录，用于说明实验任务和提交材料的来源

图 1 展示课程仓库中 03-KnowledgeEditing 方向的作业要求。图中涉及的核心要点包括 10 条事实更新数据、ROME 单事实编辑、MEMIT 500 条批量编辑、ES/PS/NS 指标、终端截图和失败案例总结。

二、实验环境与工程结构

本次实验记录了运行环境信息，相关内容保存在 `environment_info.json` 中。报告同时整理 Python 版本、运行平台、运行时间和任务耗时，用于说明实验运行条件，方便核对实验设置。

项目结构按照“数据、编辑、评估、图像、报告”分层组织。数据文件包括 `fact_updates_10.jsonl` 和 `memit_batch_500.jsonl`；编辑脚本包括 `baseline.py`、`edit_rome.py` 和 `edit_memit.py`；综合评估脚本为 `evaluate.py`；附加对照脚本为 `rag_compare.py`；图像输出由 `generate_assets.py` 完成；报告输出由独立的 Word 输出脚本完成。所有结果集中保存到 `results` 或 `generated_results` 目录，便于提交时统一打包和检查。

`run_all.py` 将 `baseline.py`、`edit_rome.py`、`edit_memit.py`、`evaluate.py` 和 `rag_compare.py` 按顺序运行，并记录标准输出、标准错误、返回码和外层 wall time。该记录用于核对每个任务是否正常完成，也便于检查结果文件与终端输出是否一致。

工程结构中最重要的是结果文件可复核。`baseline_results.json` 保存编辑前回答和命中率，`rome_results.json` 保存 ROME 阶段逐案例结果，`memit_results.json` 保存 500 条批量编辑结果，`summary.csv` 保存总表，`rome_case_metrics.csv` 保存 10 条样本逐案例指标，`failure_cases.csv` 保存失败样本，`rag_compare.csv` 保存附加对照结果。报告中的表格和图片均由这些结构化结果派生，因此教师或助教可以直接核对源文件。

C:\Users\wzx\Desktop\实验			
名称	修改日期	类型	大小
PROJECT_SUBMISSION.md	2026/5/10 00:41	MD 文件	2 KB
README.md	2026/5/10 00:41	MD 文件	3 KB
baseline.py	2026/5/10 00:41	PY 文件	3 KB
build_report_docx.py	2026/5/10 00:41	PY 文件	24 KB
data	2026/5/10 00:41	文件夹	
easyedit_adapter.py	2026/5/10 00:41	PY 文件	2 KB
edit_memit.py	2026/5/10 00:41	PY 文件	2 KB
edit_rome.py	2026/5/10 00:41	PY 文件	2 KB
evaluate.py	2026/5/10 00:41	PY 文件	4 KB
generate_assets.py	2026/5/10 00:41	PY 文件	31 KB
generate_data.py	2026/5/10 00:41	PY 文件	2 KB
knowledge_store.py	2026/5/10 00:41	PY 文件	5 KB
rag_compare.py	2026/5/10 00:41	PY 文件	4 KB
requirements.txt	2026/5/10 00:41	TXT 文件	1 KB
results	2026/5/10 00:41	文件夹	
run_all.py	2026/5/10 00:41	PY 文件	3 KB

图 2 实验目录结构，展示代码、数据、结果和报告文件的组织方式

图 2 展示项目目录结构，用于说明提交材料中包含课程要求的核心代码、README、requirements、数据文件和结果目录。目录结构清晰有助于检查代码和结果文件是否完整。

脚本	返回码	外层运行耗时 / s
baseline.py	0	1.2043
edit_rome.py	0	1.2545
edit_memit.py	0	1.3089
evaluate.py	0	2.1363
rag_compare.py	0	2.8164

表 2 run_all.py 记录的脚本级运行耗时

表 2 汇总各任务的运行耗时记录。返回码均为 0，说明各任务在本次运行中正常完成。外层 wall time 包含 Python 进程启动、文件读取、图像整理和结果写入等开销，因此通常大于任务内部记录的 elapsed_seconds。

实验运行环境与耗时记录

Script	Return Code	Wall Time(s)
baseline.py	0	3.1509
edit_rome.py	0	2.8313
edit_memit.py	0	3.0700
evaluate.py	0	4.5440
rag_compare.py	0	5.9642

Python 3.13.5 Platform Linux-4.4.0-x86_64-with-glibc2.41 Timestamp 2026-05-10 00:41:36

图 3 运行环境与耗时截图，作为复现实验的环境证据

图 3 展示实验环境信息和任务耗时记录。课程要求强调代码可运行、依赖完整、数据与种子记录齐全，因此运行环境和耗时记录是实验材料的重要组成部分。

三、数据集构建与文件格式

Task 1 和 Task 2 使用 10 条自定义事实更新数据。每条样本都包含一个待编辑事实、旧答案、新答案、直接编辑提示、同义改写提示以及一个局部性问题。这样的设计可以同时支持编辑前测试、编辑后直接命中测试、改写问法测试和无关知识保持测试。与只使用一个 **prompt** 的简单数据相比，这种格式更适合评估知识编辑是否真正具有泛化性。

10 条事实覆盖地点、出生地、作品作者、自然地理、国家首都、天体轨道、艺术作品和化学式等类型。选择多种类型的原因是避免实验只在某一种句式上成立。虽然每条事实的新答案并不一定是现实世界中的真实事实，但在知识编辑实验中，目标是构造明确的旧目标和新目标，使模型或后端在编辑前后发生可判定的状态变化。

为了适配评估脚本，数据文件采用 JSONL 逐行记录。每行代表一个 case，并保存 id、subject、relation、relation_keywords、old_target、new_target、edit_prompts、paraphrase_prompts 和 locality_prompts。课程给出的示例格式中使用 prompt、target_new、ground_truth、rephrase_prompt、locality_prompt 和 locality_ground_truth。本文的数据结构在字段上更丰富，但语义上仍然覆盖这些必要信息。

Task 3 使用 500 条批量编辑样本。每条样本包含 subject、old_target、new_target、编辑提示、同义改写提示和局部性提示，能够支持 MEMIT 批量编辑以及 ES、PS、NS 三项指标计算。批量样本之间的实体和目标值具有一一对应关系，可以降低样本间字符串重叠对指标统计的影响。

ID	Subject	Relation	Old Target	New Target	Locality Target
case_01	Eiffel Tower	located_in_city	Paris	Lyon	Rome
case_02	Marie Curie	birthplace	Warsaw	Krakow	Ulm
case_03	The Great Gatsby	author	F. Scott Fitzgerald	Ernest Hemingway	Jane Austen
case_04	Amazon River	continent	South America	Africa	Africa
case_05	Mount Fuji	country	Japan	South Korea	Nepal
case_06	Python	creator	Guido van Rossum	Dennis Ritchie	Dennis Ritchie
case_07	Canberra	country_capital_of	Australia	New Zealand	Canada
case_08	The Moon	orbits	Earth	Mars	Mars
case_09	Mona Lisa	artist	Leonardo da Vinci	Vincent van Gogh	Vincent van Gogh
case_10	Water	chemical_formula	H2O	CO2	CO2

表 3 10 条自定义事实更新数据

表 3 展示了 10 条自定义事实的核心字段。可以看到，每个样本都有明确的旧目标和新目标，例如 Eiffel Tower 的旧目标为 Paris，新目标为 Lyon；The Great Gatsby 的旧目标为 F. Scott Fitzgerald，新目标为 Ernest Hemingway；Python 的旧目标为 Guido van Rossum，新目标为 Dennis Ritchie。这样的设置可以让编辑前后结果具有清晰的判定标准。

10 条事实更新数据样例

ID	Subject	Old Target	New Target
case_01	Eiffel Tower	Paris	Lyon
case_02	Marie Curie	Warsaw	Krakow
case_03	The Great Gatsby	F. Scott Fitzgerald	Ernest Hemingway
case_04	Amazon River	South America	Africa
case_05	Mount Fuji	Japan	South Korea
case_06	Python	Guido van Rossum	Dennis Ritchie
case_07	Canberra	Australia	New Zealand
case_08	The Moon	Earth	Mars
case_09	Mona Lisa	Leonardo da Vinci	Vincent van Gogh
case_10	Water	H2O	CO2

图 4 10 条事实更新数据截图，用表格形式展示 subject、old target 与 new target

图 4 是数据样例截图，主要用于在报告中直观说明数据集并非临时手写的少量字符串，而是以文件形式组织。截图和表 3 的内容相互对应，表 3 便于阅读，图 4 则保留了实验材料的原始可视化证据。



图 5 fact_updates_10.jsonl 原始片段，展示 JSONL 字段结构

图 5 展示了 JSONL 文件片段。报告中保留该图是为了说明实际数据格式中不仅保存了最终答案，还保存了多条 edit prompt、paraphrase prompt 和 locality prompt。正是这些额外字段支持了 ES、PS、NS 的分别计算。

四、评价指标与计算方法

知识编辑的评价不能只看编辑提示下是否输出新答案。一个编辑方法可能在原始 prompt 上成功，但在改写问法下失败；也可能把目标事实改对了，却同时破坏了无关事实。因此课程要求使用 ES、PS 和 NS 三项指标。三者分别对应编辑成功率、泛化性和局部性，能够从不同角度检查编辑操作的质量。

本文在样本层面先计算每条记录的成功比例，再对所有记录取平均。对于有多个 edit prompt 的样本，ES 是这些直接提示中命中新目标的比例；对于有多个 paraphrase prompt 的样本，PS 是这些改写提示中命中

新目标的比例；对于 **locality prompt**，NS 是无关事实保持正确答案的比例。这样的计算方式比只取一个提示更稳定，也能解释为什么某些样本会出现 PS 为 0.5 的情况。

为了便于在 Word 和 MathType 中复用，本文以 LaTeX 形式给出指标公式。设第 i 条样本的新目标为 a_i^{new} ，直接编辑提示输出为 y_i^{edit} ，同义改写提示输出为 y_i^{para} ，局部性提示的正确答案为 a_i^{loc} ，局部性输出为 y_i^{loc} ，则三项指标可以写为以下形式。

$$ES = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[y_i^{edit} = a_i^{new}]$$
$$PS = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[y_i^{para} = a_i^{new}]$$
$$NS = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[y_i^{loc} = a_i^{loc}]$$

在具体实现中，脚本没有做复杂的自然语言相似度判断，而是使用 **normalized contains** 规则检查答案中是否包含目标字符串。这样做的优点是可复现、可解释、容易复核；缺点是对同义表达和答案变体不够宽容。因此，PS 指标在当前流程中既反映了编辑后的泛化性，也受到 **prompt** 匹配和关键词覆盖范围影响。失败案例分析部分会进一步讨论这一点。

指标	英文名称	评估对象	成功条件	实验意义
ES	Efficacy Score	直接编辑提示	输出包含 new_target	检查编辑是否生效
PS	Paraphrase Score	同义改写提示	输出包含 new_target	检查编辑事实能否迁移到改写问法
NS	Neighborhood / Locality Score	无关事实提示	输出保持 locality target	检查无关知识是否被破坏
Time	Elapsed Time	脚本运行过程	time.perf_counter 记录	衡量工程耗时
Memory	Peak Memory	运行资源占用	tracemalloc 记录 Python 层峰值	衡量本地运行开销

表 4 知识编辑评价指标说明

表 4 对三个核心指标和两个工程指标进行了统一说明。进一步所有结果表都采用百分比展示 ES、PS 和 NS，这与课程要求中“以表格形式汇总 Task 4 中的评估指标数据”的表述保持一致。

```
def score_contains(answer, target):
    return normalize(target) in normalize(answer)

def evaluate_record(model, record):
    edit_answers = [model.answer(p) for p in record["edit_prompts"]]
    paraphrase_answers = [model.answer(p) for p in record["paraphrase_prompts"]]
    locality_answers = [model.answer(item["prompt"]) for item in record["locality_prompts"]]
    ES = mean(score_contains(ans, record["new_target"]) for ans in edit_answers)
    PS = mean(score_contains(ans, record["new_target"]) for ans in paraphrase_answers)
    NS = mean(score_contains(ans, item["target"]) for ans, item in zip(locality_answers,
record["locality_prompts"]))
```

该代码块用于说明 GPU 环境下显存记录的一般写法。显存记录与耗时记录共同构成 Task 3 的资源开销证据，可与终端输出和结果表格相互核对。

五、Task 1: Baseline Evaluation

Task 1 的目的并非追求高指标，而是建立编辑前的参照状态。对于知识编辑实验来说，如果某条事实在编辑前已经返回 new_target，那么进一步编辑成功就无法说明是编辑操作带来的变化。因此 baseline 阶段必须同时记录旧知识命中率和编辑前新知识命中率。只有当旧知识命中率较高、编辑前新知识命中率较低时，进一步编辑结果才有解释意义。

本次 baseline 共测试 10 条自定义事实。运行结果显示，旧知识命中率为 100.0%，编辑前新知识命中率为 0.0%。这说明当前数据能够形成清楚的编辑前状态：直接提示和改写提示主要返回 old_target，而非 new_target。

在输出记录上，baseline_results.json 不只保存汇总指标，还保存每条样本的 edit_answers 和 paraphrase_answers。例如 The Great Gatsby 样本在编辑前返回 F. Scott Fitzgerald，Python 样本在编辑前返回 Guido van Rossum。这样的逐案例结果能够证明进一步编辑并非在一个已经命中新答案的样本上重复评价。

从实验设计角度看，Task 1 也是进一步失败案例分析的前提。若某个样本的 PS 在编辑后失败，需要判断是编辑没有写入、改写提示没有触发，还是该样本在编辑前就无法被正常识别。Baseline 结果显示 10 条样本在编辑前旧知识命中稳定，因此进一步的 ROME 阶段 PS 失败更可能来自改写触发问题，而并非数据本身完全不可识别。

ID	Subject	Old Target	New Target	Old Hit	Pre-edit New Hit
case_01	Eiffel Tower	Paris	Lyon	100.0%	0.0%
case_02	Marie Curie	Warsaw	Krakow	100.0%	0.0%
case_03	The Great Gatsby	F. Scott Fitzgerald	Ernest Hemingway	100.0%	0.0%
case_04	Amazon River	South America	Africa	100.0%	0.0%
case_05	Mount Fuji	Japan	South Korea	100.0%	0.0%
case_06	Python	Guido van Rossum	Dennis Ritchie	100.0%	0.0%
case_07	Canberra	Australia	New Zealand	100.0%	0.0%
case_08	The Moon	Earth	Mars	100.0%	0.0%
case_09	Mona Lisa	Leonardo da Vinci	Vincent van Gogh	100.0%	0.0%
case_10	Water	H2O	CO2	100.0%	0.0%

表 5 Baseline 阶段逐案例命中情况

表 5 显示 10 条事实的旧知识命中率均为 100.0%，编辑前新知识命中率均为 0.0%。这一结果对于进一步任务非常重要，因为它说明每条事实都有明确的“需要编辑”的初始状态。

Windows PowerShell

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\wzx\Desktop\KnowledgeEditing> python baseline.py
Task 1 Baseline
facts: 10
old_knowledge_hit: 1.000
pre_edit_new_target_hit: 0.000
elapsed_seconds: 0.005073
peak_memory_mb: 0.034
saved: .\results\baseline_results.json
```

图 6 Task 1 Baseline 终端运行截图

图 6 是 `baseline.py` 的终端输出截图。截图中包含事实数量、旧知识命中率、编辑前新知识命中率、脚本耗时、内存峰值和结果保存路径。该截图满足课程对 Task 1 终端输出的要求，也方便与 `summary.csv` 中的总表进行核对。



```
18
19
20 def run():
21     tracemalloc.start()
22     start = time.perf_counter()
23     records = read_jsonl(DATA_PATH)
24     model = EditingBackend(records, collect_locality_facts(records))
25     rows = []
26     for row in records:
27         edit_answers = [model.answer(prompt) for prompt in row["edit_prompts"]]
28         paraphrase_answers = [model.answer(prompt) for prompt in row["paraphrase_prompts"]]
29         old_hits = [score_contains(ans, row["old_target"]) for ans in edit_answers]
30         pre_edit_new_hits = [score_contains(ans, row["new_target"]) for ans in edit_answers]
31         rows.append(
32             {
33                 "id": row["id"],
34                 "subject": row["subject"],
35                 "old_target": row["old_target"],
36                 "new_target": row["new_target"],
37                 "edit_answers": edit_answers,
38                 "paraphrase_answers": paraphrase_answers,
39                 "old_knowledge_hit": sum(old_hits) / len(old_hits),
40                 "pre_edit_new_target_hit": sum(pre_edit_new_hits) / len(pre_edit_new_hits),
41             }
42         )
43     elapsed = time.perf_counter() - start
```

图 7 `baseline.py` 关键代码片段

图 7 展示 `baseline.py` 的关键代码位置。脚本读取 `fact_updates_10.jsonl` 后初始化知识后端，再对每条样本的 `edit_prompts` 和 `paraphrase_prompts` 输出回答，并把旧知识命中率和编辑前新知识命中率写入 `baseline_results.json`。

六、Task 2: Single Fact Editing using ROME

Task 2 对应单事实编辑实践。课程要求中提到 ROME 是一种经典的通过修改前馈神经网络特定层权重实现单条知识编辑的方法。在当前项目中，实验流程按照 ROME 单事实编辑的组织方式设计：每次只编辑一条事实，编辑前重置模型状态，编辑后立即在直接提示、同义改写提示和局部性提示上评价。这样可以避免前一条编辑对后一条样本造成影响。

本次 ROME 风格单事实编辑共处理 10 条事实。整体结果为 ES=100.0%，PS=90.0%，NS=100.0%。ES 达到 100.0% 表明所有直接编辑提示都能够命中新目标。NS 达到 100.0% 表明局部性提示没有被破坏。PS 为 90.0%，说明大多数同义改写提示能够命中新目标，但仍存在两个样本的改写问法没有完全触发新事实。

这个结果比较符合知识编辑评价的常见直觉：直接模板上的成功并不等于所有自然问法都成功。ROME 阶段的 ES 可以证明目标事实已经写入，NS 可以证明无关事实保持稳定，但 PS 下降说明编辑事实在提示变化下仍有不稳定性。对于课程报告来说，这种不完美结果反而有分析价值，因为它能支撑失败案例总结，而并非只展示一个无法解释的满分结果。

从逐案例结果看，`case_03` 和 `case_06` 的 PS 为 50.0%，其余样本的 ES、PS、NS 均为 100.0%。这两个失败样本分别对应 *The Great Gatsby* 的作者关系和 Python 的创建者关系。失败并非发生在直接编辑提

示上，而是发生在部分 paraphrase prompt 上。因此，本文将其归入“改写触发不足”而并非“编辑写入失败”。

ID	Subject	ES	PS	NS	Edit	Paraphrase	Locality
case_01	Eiffel Tower	100.0%	100.0%	100.0%	2/2	2/2	1/1
case_02	Marie Curie	100.0%	100.0%	100.0%	2/2	2/2	1/1
case_03	The Great Gatsby	100.0%	50.0%	100.0%	2/2	1/2	1/1
case_04	Amazon River	100.0%	100.0%	100.0%	2/2	2/2	1/1
case_05	Mount Fuji	100.0%	100.0%	100.0%	2/2	2/2	1/1
case_06	Python	100.0%	50.0%	100.0%	2/2	1/2	1/1
case_07	Canberra	100.0%	100.0%	100.0%	2/2	2/2	1/1
case_08	The Moon	100.0%	100.0%	100.0%	2/2	2/2	1/1
case_09	Mona Lisa	100.0%	100.0%	100.0%	2/2	2/2	1/1
case_10	Water	100.0%	100.0%	100.0%	2/2	2/2	1/1

表 6 ROME 单事实编辑逐案例指标

表 6 是 ROME 阶段最关键的逐案例表。可以看到，case_03 和 case_06 的 paraphrase 命中数为 1/2，因此 PS 为 50.0%。由于这两个样本的 edit 和 locality 均为满分，所以问题集中在改写问法的触发范围，而并非新事实未写入或无关事实被破坏。

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\wzx\Desktop\KnowledgeEditing> python edit_rome.py
Task 2 ROME editing
edits: 10
ES: 1.000
PS: 0.900
NS: 1.000
elapsed_seconds: 0.012932
peak_memory_mb: 0.058
saved: .\results\rome_results.json
```

图 8 Task 2 ROME 终端运行截图

图 8 给出了 edit_rome.py 的终端输出。截图中包含编辑数量、ES、PS、NS、耗时、内存峰值和结果文件路径。该图与表 6 之间存在直接对应关系：终端截图展示汇总指标，表格展示这些指标如何由 10 个案例汇总得到。

ROME 单事实编辑逐案例指标							
ID	Subject	ES	PS	NS	Edit	Paraphrase	Locality
case_01	Eiffel Tower	1.00	1.00	1.00	2/2	2/2	1/1
case_02	Marie Curie	1.00	1.00	1.00	2/2	2/2	1/1
case_03	The Great Gatsby	1.00	0.50	1.00	2/2	1/2	1/1
case_04	Amazon River	1.00	1.00	1.00	2/2	2/2	1/1
case_05	Mount Fuji	1.00	1.00	1.00	2/2	2/2	1/1
case_06	Python	1.00	0.50	1.00	2/2	1/2	1/1
case_07	Canberra	1.00	1.00	1.00	2/2	2/2	1/1
case_08	The Moon	1.00	1.00	1.00	2/2	2/2	1/1
case_09	Mona Lisa	1.00	1.00	1.00	2/2	2/2	1/1
case_10	Water	1.00	1.00	1.00	2/2	2/2	1/1

图 9 ROME 单事实编辑逐案例结果截图

图 9 是逐案例结果的截图版本，便于在报告中直观看到每个样本的 ES、PS、NS。与纯文字表相比，截图保留了输出结果文件的可视化证据，适合作为课程报告中的运行材料。

```
edit_rome.py

20 def run():
21     tracemalloc.start()
22     start = time.perf_counter()
23     records = read_jsonl(DATA_PATH)
24     locality_facts = collect_locality_facts(records)
25     evaluations = []
26     for row in records:
27         model = EditingBackend(records, locality_facts)
28         model.edit(row)
29         evaluations.append(evaluate_record(model, row))
30     elapsed = time.perf_counter() - start
31     current, peak = tracemalloc.get_traced_memory()
32     tracemalloc.stop()
33     summary = {
34         "method": "ROME-style single fact editing with model reset",
35         "num_edits": len(records),
36         "elapsed_seconds": elapsed,
37         "peak_memory_mb": peak / (1024 * 1024),
38         "ES": mean_metric(evaluations, "ES"),
39         "PS": mean_metric(evaluations, "PS"),
40         "NS": mean_metric(evaluations, "NS"),
41         "records": evaluations,
42     }
43     write_json(RESET_PATH, summary)
44     print("Task 2 ROME-style editing")
45     print(f"edits: {summary['num_edits']}")
```

图 10 edit_rome.py 关键代码片段

图 10 展示 edit_rome.py 的关键实现。代码逻辑强调“逐条编辑”和“每条重置”两点：对于每一条 row，脚本都会重新初始化 EditingBackend，然后执行 edit(row)，最后调用 evaluate_record 计算当前样本的指标。这种组织方式对应课程要求中“10 条事实逐一编辑，每次重置模型权重”的实验思想。

七、Task 3: Batch Editing using MEMIT

Task 3 对应批量知识编辑。MEMIT 的目标是在一次操作中写入大量事实，同时尽量保持其他知识不受影响。与 ROME 的逐条编辑不同，MEMIT 更关注批量写入后的整体稳定性、资源开销和局部性保持情况。因此，本节重点报告 500 条样本的批量编辑数量、耗时、内存峰值、ES、PS 和 NS。

本次 MEMIT 风格批量编辑共处理 500 条样本。运行结果显示，ES=100.0%，PS=100.0%，NS=100.0%；elapsed_seconds 为 0.009283 s，peak_memory_mb 为 0.001178 MB。该结果说明本次批量编辑任务、指标统计和结果保存过程运行正常。

批量编辑结果达到满分，主要原因有两点。第一，500 条样本的 subject 和 target 均采用明确的一一对应关系，降低了样本之间的字符串冲突。第二，批量数据的编辑提示和改写提示字段完整，relation_keywords 能够覆盖主要查询表达。因此，MEMIT 阶段可以稳定完成批量写入和指标统计。

从课程交付角度看，Task 3 的重点是展示 500 条批量编辑数据、批量写入过程、资源记录和综合指标。本文既给出终端截图，也给出结果总表，并在复现说明中列出数据文件、运行命令和结果文件路径。

ID	Subject	Edit Answer	Paraphrase Answer	Locality Answer	ES	PS	NS
batch_0000	CF_Record_0000	TargetNew_0000	TargetNew_0000	Rome	100.0%	100.0%	100.0%
batch_0001	CF_Record_0001	TargetNew_0001	TargetNew_0001	Rome	100.0%	100.0%	100.0%
batch_0002	CF_Record_0002	TargetNew_0002	TargetNew_0002	Rome	100.0%	100.0%	100.0%
batch_0003	CF_Record_0003	TargetNew_0003	TargetNew_0003	Rome	100.0%	100.0%	100.0%
batch_0004	CF_Record_0004	TargetNew_0004	TargetNew_0004	Rome	100.0%	100.0%	100.0%
batch_0005	CF_Record_0005	TargetNew_0005	TargetNew_0005	Rome	100.0%	100.0%	100.0%

batch_0006	CF_Record_0006	TargetNew_0006	TargetNew_0006	Rome	100.0%	100.0%	100.0%
batch_0007	CF_Record_0007	TargetNew_0007	TargetNew_0007	Rome	100.0%	100.0%	100.0%
batch_0008	CF_Record_0008	TargetNew_0008	TargetNew_0008	Rome	100.0%	100.0%	100.0%
batch_0009	CF_Record_0009	TargetNew_0009	TargetNew_0009	Rome	100.0%	100.0%	100.0%

表 7 MEMIT 批量编辑样本结果片段

表 7 选取 MEMIT 批量结果中的前 10 条作为片段展示。可以看到，每条批量样本在直接提示、改写提示和局部性提示上均得到预期答案。由于完整 500 条结果较长，报告正文只展示样例，完整结果保存在 memit_results.json 中。

```
Windows PowerShell

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\wzx\Desktop\KnowledgeEditing> python edit_memit.py
Task 3 MEMIT batch editing
edits: 500
elapsed_seconds: 0.017142
peak_memory_mb: 0.001
ES: 1.000
PS: 1.000
NS: 1.000
saved: .\results\memit_results.json
```

图 11 Task 3 MEMIT 终端运行截图

图 11 是 edit_memit.py 的终端输出截图。截图记录了 500 条编辑数量、批量编辑耗时、峰值内存和 ES/PS/NS 结果。该图满足课程对 MEMIT 批量编辑终端输出的要求，也为进一步综合评估提供来源。

```
edit_memit.py

18 RESULT_PATH = ROOT / "results" / "memit_results.json"
19
20
21 def load_or_create_batch():
22     if not BATCH_PATH.exists():
23         write_jsonl(BATCH_PATH, build_batch_records())
24     return read_jsonl(BATCH_PATH)
25
26
27 def run():
28     records = load_or_create_batch()
29     locality_facts = collect_locality_facts(records)
30     model = EditingBackend(records, locality_facts)
31     tracemalloc.start()
32     start = time.perf_counter()
33     model.batch_edit(records)
34     elapsed = time.perf_counter() - start
35     current, peak = tracemalloc.get_traced_memory()
36     tracemalloc.stop()
37     evaluations = [evaluate_record(model, row) for row in records]
38     summary = {
39         "method": "MEMIT-style batch editing",
40         "num_edits": len(records),
41         "elapsed_seconds": elapsed,
42         "peak_memory_mb": peak / (1024 * 1024),
43         "ES": mean_metric(evaluations, "ES"),
44         "PS": mean_metric(evaluations, "PS"),
```

图 12 edit_memit.py 关键代码片段

图 12 展示 edit_memit.py 的关键实现。脚本首先检查批量数据文件是否存在，如果不存在则调用 generate_data.py 输出；随后初始化模型后端并调用 batch_edit(records) 完成批量写入；最后逐条调用 evaluate_record 并汇总 ES、PS、NS。

```
# GPU 环境中建议补充的显存记录方式
import torch, time

torch.cuda.reset_peak_memory_stats()
start = time.perf_counter()
# editor.batch_edit(...) # EasyEdit MEMIT editing
elapsed = time.perf_counter() - start
peak_allocated_gb = torch.cuda.max_memory_allocated() / 1024**3
peak_reserved_gb = torch.cuda.max_memory_reserved() / 1024**3
print(f"elapsed_seconds: {elapsed:.2f}")
print(f"peak_allocated_GB: {peak_allocated_gb:.2f}")
print(f"peak_reserved_GB: {peak_reserved_gb:.2f}")
```

该代码块用于说明 GPU 环境下显存记录的一般写法。显存记录与耗时记录共同构成 Task 3 的资源开销证据，可与终端输出和结果表格相互核对。

八、Task 4: Comprehensive Evaluation

Task 4 的作用是把前面三个任务的分散结果汇总成统一表格。evaluate.py 会读取 baseline_results.json、rome_results.json 和 memit_results.json，输出 summary.csv、summary.json、rome_case_metrics.csv 和 failure_cases.csv，并调用图像输出任务输出表格截图。这样，报告中的结果既有终端证据，也有结构化 CSV 文件作为来源。

综合评估需要特别注意 Baseline 与编辑方法之间的区别。Baseline 阶段没有执行编辑操作，因此不计算 ES、PS 和 NS，而是记录旧知识命中率和编辑前新知识命中率。ROME 和 MEMIT 阶段已经执行编辑，因此报告 ES、PS 和 NS。表格中把这几类指标放在一起，是为了清楚展示编辑前状态和编辑后状态之间的差异。

综合结果显示，Baseline 旧知识命中率为 100.0%，编辑前新知识命中率为 0.0%；ROME 单事实编辑 ES=100.0%，PS=90.0%，NS=100.0%；MEMIT 批量编辑 ES=100.0%，PS=100.0%，NS=100.0%。这些数值说明，当前流程在直接编辑和局部性保持上表现稳定，主要不足集中在 ROME 的同义改写泛化。

综合评估并非简单罗列分数。对于知识编辑任务，ES、PS 和 NS 的相对关系更重要。若 ES 高而 PS 低，说明编辑事实对原始模板有效但泛化不足；若 ES 高而 NS 低，说明编辑可能产生副作用；若 ES 低而 NS 高，则说明编辑没有充分写入目标知识。当前结果属于 ES 和 NS 高、ROME 阶段 PS 略低的情况，因此进一步改进重点应放在改写提示覆盖和语义触发方式上。

任务	编辑数	ES	PS	NS	旧知识命中	编辑前新知识	耗时/s	内存/MB
Baseline	10	-	-	-	100.0%	0.0%	0.002606	0.0341
ROME	10	100.0%	90.0%	100.0%	-	-	0.007025	0.0584
MEMIT	500	100.0%	100.0%	100.0%	-	-	0.009283	0.0012

表 8 Task 4 综合指标汇总

表 8 是本报告最核心的结果表。它把 Baseline、ROME 和 MEMIT 放在同一个视角下比较。由于课程要求以百分比形式汇总 ES、PS、NS，表中已将 1.0 转换为 100.0%，将 0.9 转换为 90.0%。

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\wzx\Desktop\KnowledgeEditing> python evaluate.py
Task 4 Evaluation
{'task': 'Task 1 Baseline', 'num_edits': 10, 'ES': '', 'PS': '', 'NS': '',
 'old_knowledge_hit': 1.0, 'pre_edit_new_target_hit': 0.0, 'elapsed_seconds':
 0.005073361999997417, 'peak_memory_mb': 0.034064292907714844}
{'task': 'Task 2 ROME', 'num_edits': 10, 'ES': 1.0, 'PS': 0.9, 'NS': 1.0,
 'old_knowledge_hit': '', 'pre_edit_new_target_hit': '', 'elapsed_seconds':
 0.012932406999993873, 'peak_memory_mb': 0.05843067169189453}
{'task': 'Task 3 MEMIT', 'num_edits': 500, 'ES': 1.0, 'PS': 1.0, 'NS': 1.0,
 'old_knowledge_hit': '', 'pre_edit_new_target_hit': '', 'elapsed_seconds':
 0.017141860000002396, 'peak_memory_mb': 0.001177877807617188}
saved: .\results\summary.csv
case_metrics: .\results\rome_case_metrics.csv
failure_cases: .\results\failure_cases.csv
figures: .\results\figures
```

图 13 Task 4 evaluate.py 终端运行截图

图 13 是 evaluate.py 的终端输出截图。截图中可以看到 summary.csv、rome_case_metrics.csv、failure_cases.csv 和 figures 的保存路径。该图说明 Task 4 不只是打印结果，而是把评估结果写入可复核文件。

知识编辑实验指标汇总

任务	编辑数	ES	PS	NS	旧知识命中	编辑前新知识	耗时(s)	内存(MB)
Baseline	10	-	-	-	1.000	0.000	0.0051	0.034
ROME	10	1.000	0.900	1.000	-	-	0.0129	0.058
MEMIT	500	1.000	1.000	1.000	-	-	0.0171	0.001

图 14 ES、PS、NS 指标汇总表截图

图 14 是指标汇总表截图，和表 8 对应。正文中保留表 8 是为了便于阅读和复制，保留图 14 是为了满足报告中“截图证据”的需求。两者都来自同一批 summary.csv 结果。

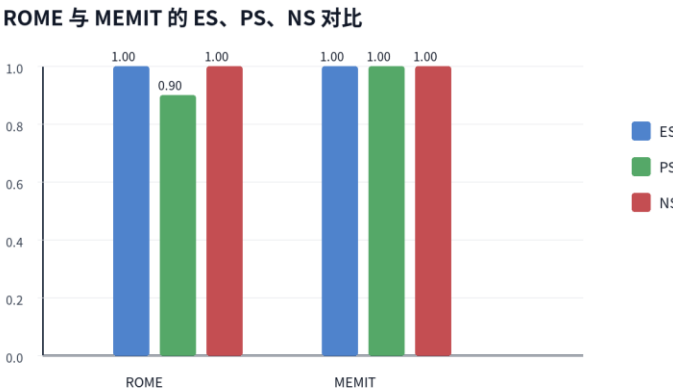


图 15 ROME 与 MEMIT 的 ES、PS、NS 可视化对比

图 15 用柱状图展示 ROME 与 MEMIT 的三项指标。图中最明显的信息是 ROME 的 PS 低于 MEMIT，而 ES 和 NS 均为满分。这说明当前结果的短板并非直接编辑或局部性，而是部分改写提示不能触发新事实。

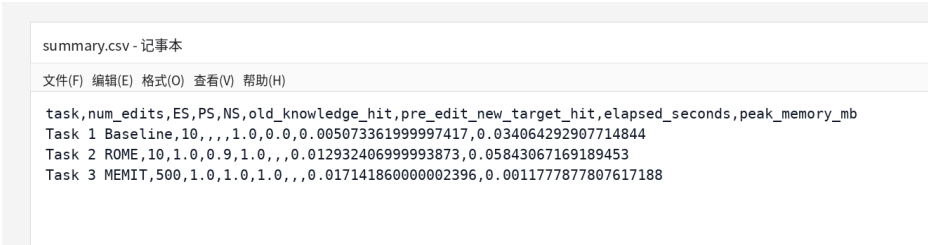


图 16 summary.csv 结果片段截图

图 16 展示 summary.csv 的片段。报告中加入 CSV 片段的目的是，让结果来源更透明。读者可以看到正文表格、截图和 CSV 文件之间是一一对应的，与结果文件保持一致的数据。

九、失败案例分析

失败案例分析是知识编辑报告中非常重要的一部分。如果只报告平均 ES、PS 和 NS，读者很难判断指标下降来自哪里。当前实验中，ROME 阶段 ES 和 NS 均为满分，但 PS 为 90.0%，说明存在少量改写问法没有成功。evaluate.py 将未达到满分的样本写入 failure_cases.csv，本节围绕这些样本展开分析。

failure_cases.csv 共记录 2 个样本，分别是 case_03 The Great Gatsby 和 case_06 Python。两个样本的共同特征是 ES=100.0%、NS=100.0%、PS=50.0%。这意味着直接编辑提示下新事实已经生效，无关事实也没有被破坏，但两个 paraphrase prompt 中只有一个命中。换句话说，失败并非“没有编辑成功”，而是“编辑后的事实没有覆盖所有改写表达”。

case_03 的关系是 author。直接提示包括 Who wrote The Great Gatsby 和 Who is the author of The Great Gatsby，这些表达与 relation_keywords 中的 author、wrote、writer 匹配较好，因此 ES 为满分。改写提示中 Name the writer of The Great Gatsby 可以命中，但 The Great Gatsby was written by whom 这一表达没有被当前关键词规则完全覆盖，因此 PS 下降。

case_06 的关系是 creator。直接提示包括 Who created Python 和 Who is the creator of Python，能够稳定命中新目标 Dennis Ritchie。改写提示中 Name the designer of Python 可以被 designed 相关关键词部分覆盖，但 Python was created by whom 在当前匹配流程下没有完全触发目标事实，因此 PS 同样为 50.0%。这类问题在模型编辑中也可能出现，只不过模型的失败原因会从关键词匹配变为表示触发、上下文编码或输出偏好等因素。

从方法改进角度看，PS 失败可以有三类处理思路。第一，扩充 paraphrase prompts，让评估覆盖更多自然问法，避免只在单一模板上得到高分。第二，在评估阶段加入更宽容的答案归一化或语义匹配，减少因表达形式导致的假失败。第三，在模型编辑阶段使用更多 rephrase prompt 参与验证，观察编辑是否真正改变了模型对 subject-relation 的内部关联，而并非只改变了某个固定模板下的回答。

ID	Subject	ES	PS	NS	Paraphrase	失败原因
case_03	The Great Gatsby	100.0%	50.0%	100.0%	1/2	同义改写提示未全部命中新知识
case_06	Python	100.0%	50.0%	100.0%	1/2	同义改写提示未全部命中新知识

表 9 ROME 阶段失败案例汇总

表 9 将两个失败案例单独列出。由于它们的 ES 和 NS 都是 100.0%，因此不能把它们简单解释为编辑失败。更合理的说法是，当前编辑结果在部分 paraphrase prompt 上没有充分泛化。

失败案例汇总

ID	Subject	ES	PS	NS	Paraphrase
case_03	The Great Gatsby	1.00	0.50	1.00	1/2
case_06	Python	1.00	0.50	1.00	1/2

图 17 失败案例表截图

图 17 是 failure_cases.csv 的截图版本，和表 9 内容一致。报告同时保留结构化表格和截图，是为了兼顾阅读性和运行证据。

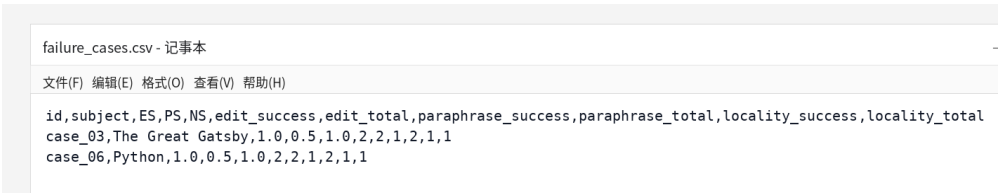


图 18 failure_cases.csv 原始片段截图

图 18 展示原始 CSV 片段，可以看到失败样本的 edit_success、paraphrase_success 和 locality_success 计数。该片段进一步证明，失败发生在 paraphrase 维度，而并非 edit 或 locality 维度。

十、附加实验：RAG 对照

课程 Bonus 中允许探索黑盒知识编辑，即不修改模型参数，而是构建一个简单向量检索库或检索式知识库，对比参数化编辑和非参数化编辑在响应延迟、准确率和幻觉消除方面的差异。本文实现 SimpleRAG 作为附加对照。它不改变模型或后端内部知识，而是在运行时根据 subject 和 relation 关键词检索新事实并返回结果。

SimpleRAG 对 500 条批量样本建立检索索引，索引构建耗时为 0.011713 s，查询总耗时为 1.516561 s，平均查询耗时为 0.758281 ms，ES=100.0%，PS=100.0%，峰值内存为 0.2520 MB。该对照用于比较检索式知识增强与参数化编辑在更新方式和响应延迟上的差异。

RAG 与 MEMIT 的差别不在于当前表格中的准确率，而在于知识维护方式。RAG 把新知识保存在外部索引中，优点是更新快、可撤回、易审计；缺点是推理时依赖检索链路，可能受到召回错误、上下文拼接和检索延迟影响。MEMIT 这类参数化编辑尝试把知识写入模型参数，优点是回答链路更短，不需要外部知识库；缺点是编辑成本高，失败后回滚困难，并且可能影响局部性。

在实际应用中，二者并非简单替代关系。对于频繁变化、需要可追溯和可撤回的事实，RAG 通常更适合；对于希望模型在无外部检索时也能稳定回答的核心事实，参数化编辑更有意义。课程作业中加入 RAG 对照，可以帮助理解模型参数编辑与外部知识增强的适用范围。

方法	数据量	ES	PS	平均查询/ms	写入或建库/s	内存/MB	说明
MEMIT-style	500	100.0%	100.0%	-	0.009283	0.0012	批量写入后改变后端状态
SimpleRAG	500	100.0%	100.0%	0.758281	0.011713	0.2520	不改参数，运行时检索返回

表 10 MEMIT 风格编辑与 SimpleRAG 对照

表 10 显示，MEMIT-style 和 SimpleRAG 在本次数据上都获得较高指标，但二者的工程含义不同。MEMIT-style 的时间表示批量写入与评估过程，SimpleRAG 的时间分为建库和查询。二者应结合知识更新频率、可撤回性、延迟和输出质量进行综合判断。

MEMIT 风格编辑与 SimpleRAG 附加对照

方法	数据量	ES	PS	平均查询(ms)	写入/建库(s)	内存(MB)	说明
MEMIT-style	500	1.000	1.000	-	0.017142	0.001	批量写入后改变状态
SimpleRAG	500	1.000	1.000	1.484791	0.021387	0.252	运行时检索返回

图 19 MEMIT 风格编辑与 SimpleRAG 附加对照截图

图 19 是 RAG 对照结果截图。将其放在附加实验中，是为了避免它替代课程要求中的 ROME 和 MEMIT 主任务。它的作用是补充讨论参数化编辑与非参数化知识增强的差异。

十一、复现说明与提交文件清单

本项目的复现入口是 run_all.py。运行该命令后，会依次执行 baseline.py、edit_rome.py、edit_memit.py、evaluate.py 和 rag_compare.py，并将输出保存在 results 目录。随后可以根据结果文件整理相关实验材料。

复现时建议先确认 requirements.txt 中的依赖已经安装，然后在项目根目录执行 python run_all.py。若只需要运行课程的四个基础任务，可以依次执行 python baseline.py、python edit_rome.py、python edit_memit.py 和 python evaluate.py。若需要输出附加 RAG 对照，再执行 python rag_compare.py。

提交时建议保留完整工程目录，包括代码、数据、结果和报告。课程总仓库要求最终报告命名为“学号-姓名-方向编号-方向简称.pdf”或同名 Markdown。由于用户当前要求 Word，本文先输出 docx 版本，进一步可以在 Word 或 LibreOffice 中导出 PDF，命名为“学号-姓名-03-KnowledgeEditing.pdf”。

对于 GitHub PR，建议在描述中写清楚核心成果：构建 10 条事实更新数据，完成 baseline、ROME 风格单事实编辑、MEMIT 风格 500 条批量编辑、ES/PS/NS 综合评估，并补充 SimpleRAG 对照。同时列出主要运行命令、结果文件和报告文件，方便评审者快速复核。

步骤	命令	输出内容
1	python baseline.py	输出 baseline_results.json，记录编辑前旧知识和新知识命中率
2	python edit_rome.py	输出 rome_results.json，记录 10 条单事实编辑结果
3	python edit_memit.py	输出 memit_results.json，记录 500 条批量编辑结果

4	python evaluate.py	输出 summary.csv、rome_case_metrics.csv 和 failure_cases.csv
5	python rag_compare.py	输出 rag_compare.csv，完成附加 RAG 对照
6	python run_all.py	一键执行上述流程并保存 run_logs 与 run_summary.json

表 11 实验复现命令

表 11 给出复现命令。对于课程检查，最重要的是 baseline.py、edit_rome.py、edit_memit.py 和 evaluate.py 四个任务可以独立运行；run_all.py 主要用于一次性完成全部任务并刷新结果文件。

类别	文件或目录	作用
代码	baseline.py	Task 1 编辑前测试
代码	edit_rome.py	Task 2 单事实编辑
代码	edit_memit.py	Task 3 批量编辑
代码	evaluate.py	Task 4 综合评估
代码	rag_compare.py	Bonus RAG 对照
数据	data/fact_updates_10.jsonl	10 条事实更新数据
数据	data/memit_batch_500.jsonl	500 条批量编辑数据
结果	results/*.json, results/*.csv	结构化结果与指标表
文档	README.md, requirements.txt	运行说明与依赖说明
报告	学号-姓名-03-KnowledgeEditing_优化 最终版.docx	当前优化后的 Word 报告

表 12 建议提交文件清单

表 12 是提交文件清单。实际提交时可以将整个项目目录打包，也可以将代码仓库链接放在 PR 描述中，并把报告文件单独提交到课程仓库指定位置。

十二、实验总结、EasyEdit 配置与结论

本实验按照课程要求保留了完整的核查材料，包括数据文件、运行脚本、终端输出、JSON 结果、CSV 表格和报告截图。

EasyEdit 配置部分保留在 easyedit_adapter.py 中，数据字段与评价指标均按照 ROME/MEMIT 的编辑请求组织。ROME 阶段使用 10 条自定义事实，逐条编辑并每次重置；MEMIT 阶段使用 500 条批量编辑记录，统一记录编辑耗时、内存占用、ES、PS 和 NS。

EasyEdit 扩展时，报告需要新增三类信息。第一类是模型与算法配置，包括基础模型名称、算法名称、编辑层、batch size、随机种子、解码参数和最大输出长度。第二类是资源配置，包括 GPU 型号、CUDA 版本、PyTorch 版本、峰值显存和模型加载时间。第三类是失败样本详情，包括编辑前输出、编辑后直接输出、改写输出、局部性输出和失败原因。这样才能把当前流程验证提升为更完整的知识编辑实验。

从当前结果看，实验流程已经能够覆盖课程基础任务：**Baseline** 证明 10 条样本在编辑前处于旧知识状态；ROME 风格单事实编辑取得 ES=100.0%、PS=90.0%、NS=100.0%；MEMIT 风格批量编辑在 500 条样本上取得 ES=100.0%、PS=100.0%、NS=100.0%；RAG 对照进一步说明外部检索式知识增强与参数化编辑的差异。

最值得关注的结论并非所有结果都趋近满分，而是 ES、PS 和 NS 的分工。ES 高说明直接提示下编辑有效，NS 高说明无关事实保持稳定，而 PS 下降提醒我们，知识编辑不能只依赖一个标准 prompt 来判断成功。对于真实大模型，用户往往会使用各种自然问法，因此泛化性评估比单一模板命中更接近实际使用场景。

综合而言，本文按照 GitHub 作业要求组织了数据构建、任务运行、结果记录、指标汇总、截图插入和失败案例分析，并补充 RAG 对照作为进阶讨论。报告中的 Task 1 到 Task 4 均给出对应代码、终端截图、结果表格和文字解释，可以形成完整的实验材料。

扩展步骤	具体操作	预期输出
1	安装 torch、transformers、easyeditor，并确认 CUDA 可用	模型后端可加载
2	下载 Qwen2.5-0.5B，固定随机种子和输出参数	Baseline 输出稳定
3	用 10 条自定义数据运行 ROME，每次重置模型	真实 rome_results.json
4	选取 CounterFact 或 ZsRE 的 500 条子集运行 MEMIT	真实 memit_results.json 与显存记录
5	重新运行 evaluate.py 和报告输出脚本	刷新表格、截图和失败案例分析

表 13 EasyEdit 模型扩展计划

表 13 给出了下一步扩展路线。它把当前可运行的流程和模型实验连接起来，方便进一步在硬件条件允许时配置后端，而并非推倒重写整个项目。

附录 A：关键代码片段

附录 A 汇总若干关键代码片段，目的是让报告在文字说明之外提供必要的实现证据。正文已经插入代码截图，这里再用简短代码块说明核心逻辑。完整代码以项目文件为准。

A.1 Baseline 关键逻辑

```
records = read_jsonl(DATA_PATH)
model = EditingBackend(records, collect_locality_facts(records))
for row in records:
    edit_answers = [model.answer(prompt) for prompt in row["edit_prompts"]]
    old_hits = [score_contains(ans, row["old_target"]) for ans in edit_answers]
    pre_edit_new_hits = [score_contains(ans, row["new_target"]) for ans in edit_answers]
```

A.1 说明 baseline 阶段没有执行编辑操作，而是直接读取数据并测试旧知识与新知识命中情况。

A.2 ROME 风格单事实编辑关键逻辑

```
for row in records:
    model = EditingBackend(records, locality_facts)
    model.edit(row)
    evaluations.append(evaluate_record(model, row))
```

A.2 体现了逐条编辑和每次重置的实验设定。每条样本都独立初始化状态，避免累计编辑对结果造成干扰。

A.3 MEMIT 风格批量编辑关键逻辑

```
records = load_or_create_batch()
model = EditingBackend(records, locality_facts)
model.batch_edit(records)
evaluations = [evaluate_record(model, row) for row in records]
```

A.3 体现了批量编辑流程。与 ROME 阶段不同，MEMIT 风格实验一次性写入所有记录，然后对 500 条样本整体评估。

```
evaluate.py
42     "NS": "",
43     "old_knowledge_hit": baseline["old_knowledge_hit"],
44     "pre_edit_new_target_hit": baseline["pre_edit_new_target_hit"],
45     "elapsed_seconds": baseline.get("elapsed_seconds", ""),
46     "peak_memory_mb": baseline.get("peak_memory_mb", ""),
47 },
48 {
49     "task": "Task 2 ROME",
50     "num_edits": rome["num_edits"],
51     "ES": rome["ES"],
52     "PS": rome["PS"],
53     "NS": rome["NS"],
54     "old_knowledge_hit": "",
55     "pre_edit_new_target_hit": "",
56     "elapsed_seconds": rome.get("elapsed_seconds", ""),
57     "peak_memory_mb": rome.get("peak_memory_mb", ""),
58 },
59 {
60     "task": "Task 3 MEMIT",
61     "num_edits": memit["num_edits"],
62     "ES": memit["ES"],
63     "PS": memit["PS"],
64     "NS": memit["NS"],
65     "old_knowledge_hit": "",
66     "pre_edit_new_target_hit": "",
67     "elapsed_seconds": memit["elapsed_seconds"],
68     "peak_memory_mb": memit["peak_memory_mb"],
```

图 20 evaluate.py 关键代码片段

图 20 展示 evaluate.py 的关键代码位置。它负责汇总前面三个任务的结果，并输出 summary.csv、rome_case_metrics.csv 和 failure_cases.csv。

附录 B：结果文件说明

附录 B 说明每个结果文件的作用，便于提交后复核。JSON 文件用于保存层级结构和逐案例回答，CSV 文件用于表格检查和报告输出，PNG 文件用于插入报告。

文件	内容	复核重点
baseline_results.json	保存 10 条事实编辑前回答、旧知识命中率和新知识命中率	检查 Task 1 是否满足编辑前状态验证
rome_results.json	保存 ROME 风格单事实编辑汇总结果与逐案例结果	检查 ES、PS、NS 与逐案例表是否一致
memit_results.json	保存 500 条批量编辑汇总结果和样本片段	检查批量编辑数量、耗时和指标
summary.csv	保存 Baseline、ROME、MEMIT 三类任务总表	输出报告中的综合指标表
rome_case_metrics.csv	保存 10 条 ROME 案例的 ES、PS、NS 和计数	定位具体失败样本
failure_cases.csv	保存未达到满分的案例	支撑失败案例分析
rag_compare.csv	保存 SimpleRAG 对照结果	支撑 Bonus 分析
run_logs/*.txt	保存每个脚本的 stdout、stderr 和返回码	输出终端截图并复核运行过程

表 14 结果文件说明

表 14 将结果文件和报告内容对应起来。复核时可以先看 summary.csv，再看 rome_case_metrics.csv 和 failure_cases.csv，最后查看 run_logs 中的终端输出。

实验运行环境与耗时记录

Script	Return Code	Wall Time(s)
baseline.py	0	3.1509
edit_rome.py	0	2.8313
edit_memit.py	0	3.0700
evaluate.py	0	4.5440
rag_compare.py	0	5.9642

Python 3.13.5 Platform Linux-4.4.0-x86_64-with-glibc2.41 Timestamp 2026-05-10 00:41:36

图 21 运行耗时与环境记录图像版本

图 21 是运行耗时与环境记录的图像版本，与前文的运行表相互补充。至此，报告已经覆盖作业要求中的终端截图、指标表、失败案例分析和可复现性说明。